Technical Design Document & API Specification

Task Management System (NestJS, TypeORM, MySQL, Class-Validator)

Disusun oleh:

No	Nama	NIM
1	Giska Aura Muhammad Prasetyo	24090055
2	Hanif Azhar	24090030
3	Akbar Rizqi Ainul Yaqin	24090060
4	Maula Adiba Mufadol	24090052

Ringkasan Dokumen

Dokumen ini menjelaskan cara kami merancang backend Task Management System, yaitu aplikasi yang membantu tim membagi tugas, memantau progres, dan berdiskusi pada setiap tugas. Isinya mencakup rancangan database, daftar lengkap endpoint API, dan panduan menjalankan proyek di komputer sendiri.

Cara membaca dokumen ini:

- **Bab 1** memberi gambaran umum sistem dan teknologi yang dipakai.
- **Bab 2** menjelaskan struktur data: tabel, relasi, dan diagram ERD.
- **Bab 3** berisi daftar endpoint API beserta contoh request dan response.
- **Bab 4** memandu langkah demi langkah dari instalasi sampai aplikasi berjalan.

1. Pendahuluan & Arsitektur Sistem

1.1 Gambaran Umum

Task Management System adalah aplikasi backend (REST API) yang membantu tim mengatur pekerjaan dengan rapi. Bayangkan sebuah tim kerja: tim itu punya beberapa anggota dan beberapa proyek, setiap proyek berisi daftar tugas, dan setiap tugas dapat diberikan kepada satu anggota.

Anggota tim juga bisa saling berdiskusi lewat komentar pada setiap tugas. Dalam sistem, kelima unsur ini disebut **Team**, **Member**, **Project**, **Task**, dan **Comment**.

1.2 Tujuan dan Lingkup

- Mengelola data Teams, Projects, Members, dan Tasks (tambah, lihat, ubah, hapus), serta menambah dan melihat Comments.
- Menjaga data tetap konsisten: antartabel dihubungkan dengan foreign key, dan aturan penghapusan sudah ditentukan (cascade atau set null).
- Memeriksa setiap data yang masuk dengan validasi DTO (Class-Validator), sehingga data yang salah ditolak sebelum disimpan.

- Memudahkan pencarian tugas: tugas dapat disaring berdasarkan status, prioritas, dan penerima tugas, lalu dibagi per halaman (pagination).
- Belum mencakup: login dan otorisasi (JWT), notifikasi, serta upload file.

1.3 Tech Stack

Komponen	Teknologi	Peran
Framework	NestJS 10+ (Node.js 18+ / TypeScript)	Struktur modular, Dependency Injection, routing
ORM	TypeORM 0.3+	Pemetaan entitas ke tabel, relasi, migrasi
Database	MySQL 8.x (driver `mysql2`)	Penyimpanan data relasional
Validasi	class-validator & class-transformer	Validasi dan transformasi DTO request
Konfigurasi	@nestjs/config	Pembacaan variabel lingkungan `.env`

1.4 Arsitektur Sistem

Aplikasi disusun dalam beberapa lapisan yang masing-masing punya tugas jelas, lalu dikelompokkan per modul NestJS agar mudah dirawat:

1. **Controller** menerima permintaan HTTP dari pengguna, memeriksa datanya lewat `ValidationPipe`, lalu mengirim jawaban.
2. **Service** menjalankan aturan bisnis, misalnya memastikan anggota yang diberi tugas memang berasal dari tim pemilik proyek.
3. **Repository (TypeORM)** membaca dan menulis data ke MySQL.
4. **Entity dan DTO**: Entity menggambarkan bentuk tabel, sedangkan DTO menetapkan format dan aturan data yang boleh dikirim.

Alur sederhananya: `Client → Controller (ValidationPipe) → Service → Repository → MySQL`. Jawaban kembali ke pengguna lewat jalur yang sama.

Struktur direktori:

```
task-management-system/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   ├── common/
│   │   ├── enums/          (member-role, task-status, task-priority)
│   │   └── filters/        (http-exception.filter.ts)
│   ├── teams/              (module, controller, service, entity, dto/)
│   ├── projects/           (module, controller, service, entity, dto/)
│   ├── members/            (module, controller, service, entity, dto/)
│   ├── tasks/              (module, controller, service, entity, dto/)
│   ├── comments/           (module, controller, service, entity, dto/)
│   └── database/
```

```
│       ├── data-source.ts
│       └── migrations/
├── .env
├── package.json
└── tsconfig.json
```

1.5 Istilah Penting

Beberapa istilah teknis berikut sering muncul di dokumen ini.

Istilah	Artinya
REST API	Cara aplikasi lain (misalnya Postman atau aplikasi mobile) berkomunikasi dengan server lewat alamat URL dan metode HTTP seperti GET dan POST
Endpoint	Satu alamat URL pada API untuk satu aksi, misalnya `GET /teams` untuk melihat daftar team
Entity	Kelas TypeScript yang mewakili satu tabel di database
DTO (Data Transfer Object)	Bentuk data yang diterima API dari pengguna, lengkap dengan aturan validasinya
ORM (TypeORM)	Alat yang memungkinkan database diakses lewat kode TypeScript tanpa menulis SQL secara manual
Foreign Key (FK)	Kolom yang menunjuk ke data di tabel lain, sebagai penghubung antartabel
Cascade	Jika data induk dihapus, data anak yang terhubung ikut terhapus
Migrasi	Catatan perubahan struktur database yang dapat dijalankan secara teratur dan berulang
Pagination	Pembagian hasil yang banyak menjadi beberapa halaman

2. Perancangan Database & ERD

2.1 Ringkasan Entitas

Database bernama `task_management` terdiri dari lima tabel: `teams`, `projects`, `members`, `tasks`, dan `comments`. Setiap tabel memakai primary key `INT UNSIGNED AUTO_INCREMENT` (nomor urut otomatis) dan memiliki kolom `created_at` serta `updated_at` bertipe `DATETIME` yang terisi otomatis lewat `@CreateDateColumn` dan `@UpdateDateColumn`.

2.2 Detail Tabel

a. Tabel `teams`

Menyimpan kelompok kerja. Tabel ini adalah induk dari `projects` dan `members`.

Kolom	Tipe	Constraint	Keterangan
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identitas unik team
name	VARCHAR(100)	NOT NULL, UNIQUE	Nama team
description	TEXT	NULL	Deskripsi team
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Waktu dibuat
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Waktu diperbarui

b. Tabel projects

Menyimpan proyek milik sebuah team.

Kolom	Tipe	Constraint	Keterangan
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identitas unik project
name	VARCHAR(150)	NOT NULL	Nama project
description	TEXT	NULL	Deskripsi project
team_id	INT UNSIGNED	FK → teams.id, NOT NULL, ON DELETE CASCADE	Team pemilik project
created_at	DATETIME	NOT NULL	Waktu dibuat
updated_at	DATETIME	NOT NULL	Waktu diperbarui

c. Tabel members

Menyimpan anggota yang tergabung dalam sebuah team.

Kolom	Tipe	Constraint	Keterangan
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identitas unik member
name	VARCHAR(100)	NOT NULL	Nama lengkap
email	VARCHAR(150)	NOT NULL, UNIQUE	Alamat email
role	ENUM('admin','manager','member')	NOT NULL, DEFAULT 'member'	Peran dalam team
team_id	INT UNSIGNED	FK → teams.id, NOT NULL, ON DELETE CASCADE	Team tempat member bergabung

Kolom	Tipe	Constraint	Keterangan
created_at	DATETIME	NOT NULL	Waktu dibuat
updated_at	DATETIME	NOT NULL	Waktu diperbarui

d. Tabel tasks

Menyimpan tugas dalam sebuah project.

Kolom	Tipe	Constraint	Keterangan
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identitas unik task
title	VARCHAR(200)	NOT NULL	Judul tugas
description	TEXT	NULL	Detail tugas
status	ENUM('todo','in_progress','review','done')	NOT NULL, DEFAULT 'todo'	Status pengerjaan
priority	ENUM('low','medium','high','urgent')	NOT NULL, DEFAULT 'medium'	Tingkat prioritas
due_date	DATE	NULL	Tenggat waktu
project_id	INT UNSIGNED	FK → projects.id, NOT NULL, ON DELETE CASCADE	Project induk
assigned_member_id	INT UNSIGNED	FK → members.id, NULL, ON DELETE SET NULL	Member penerima tugas
created_at	DATETIME	NOT NULL	Waktu dibuat
updated_at	DATETIME	NOT NULL	Waktu diperbarui

e. Tabel comments

Menyimpan komentar member pada sebuah task.

Kolom	Tipe	Constraint	Keterangan
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identitas unik komentar
content	TEXT	NOT NULL	Isi komentar
task_id	INT UNSIGNED	FK → tasks.id, NOT NULL, ON DELETE CASCADE	Task yang dikomentari
member_id	INT	FK → members.id, NOT NULL,	Penulis komentar

Kolom	Tipe	Constraint	Keterangan
	UNSIGNED	ON DELETE CASCADE	
created_at	DATETIME	NOT NULL	Waktu dibuat
updated_at	DATETIME	NOT NULL	Waktu diperbarui

2.3 Relasi Antar Tabel

No	Relasi	Kardinalitas	Foreign Key	Penjelasan
1	Team → Project	One-to-Many	projects.team_id	Satu team memiliki banyak project; setiap project milik satu team (Many-to-One dari sisi Project)
2	Team → Member	One-to-Many	members.team_id	Satu team beranggotakan banyak member; setiap member tergabung pada satu team
3	Project → Task	One-to-Many	tasks.project_id	Satu project berisi banyak task; setiap task milik satu project
4	Member → Task	One-to-Many (opsional)	tasks.assigned_member_id	Satu member dapat menerima banyak task; sebuah task boleh belum ditugaskan (NULL)
5	Task → Comment	One-to-Many	comments.task_id	Satu task memiliki banyak komentar; setiap komentar milik satu task
6	Member → Comment	One-to-Many	comments.member_id	Satu member dapat menulis banyak komentar; setiap komentar ditulis satu member

2.4 Aturan Integritas Referensial

- Jika sebuah **Team** dihapus, semua Project dan Member miliknya ikut terhapus (CASCADE), termasuk Task dan Comment yang terkait.
- Jika sebuah **Project** dihapus, semua Task di dalamnya beserta Comment-nya ikut terhapus (CASCADE).
- Jika sebuah **Member** dihapus, Task miliknya tidak ikut hilang tetapi menjadi belum ditugaskan (assigned_member_id menjadi NULL, SET NULL), sedangkan Comment yang ia tulis ikut terhapus (CASCADE).
- Jika sebuah **Task** dihapus, semua Comment-nya ikut terhapus (CASCADE).
- Aturan bisnis di Service: Member yang ditugaskan ke Task, maupun Member yang menulis Comment, harus berasal dari Team yang sama dengan Project tempat Task berada. Jika tidak, API menjawab 400 Bad Request.

2.5 Entity Relationship Diagram (Mermaid)

```
erDiagram
    TEAMS ||--o{ PROJECTS : "memiliki"
    TEAMS ||--o{ MEMBERS : "beranggotakan"
    PROJECTS ||--o{ TASKS : "berisi"
    MEMBERS |o--o{ TASKS : "ditugaskan"
    TASKS ||--o{ COMMENTS : "memiliki"
    MEMBERS ||--o{ COMMENTS : "menulis"

    TEAMS {
        int id PK
        varchar name UK
        text description
        datetime created_at
        datetime updated_at
    }
    PROJECTS {
        int id PK
        varchar name
        text description
        int team_id FK
        datetime created_at
        datetime updated_at
    }
    MEMBERS {
        int id PK
        varchar name
        varchar email UK
        enum role
        int team_id FK
        datetime created_at
        datetime updated_at
    }
    TASKS {
        int id PK
        varchar title
        text description
        enum status
        enum priority
        date due_date
        int project_id FK
        int assigned_member_id FK
        datetime created_at
        datetime updated_at
    }
    COMMENTS {
```

```
    int id PK
    text content
    int task_id FK
    int member_id FK
    datetime created_at
    datetime updated_at
}
```

3. Spesifikasi API (API Specification & Endpoints)

3.1 Konvensi Umum

- **Base URL**: http://localhost:3000/api/v1
- **Content-Type**: application/json untuk request maupun response.
- **Penamaan field JSON**: snake_case, sama dengan nama kolom di database.
- **Format tanggal**: created_at dan updated_at memakai ISO 8601 (2026-11-12T08:30:00.000Z), sedangkan due_date memakai YYYY-MM-DD.
- **Pembaruan data**: memakai PATCH (partial update), jadi cukup kirim field yang ingin diubah. Semua field pada DTO update bersifat opsional.
- **ValidationPipe global** (main.ts): whitelist: true, forbidNonWhitelisted: true, dan transform: true. Field di luar DTO ditolak dengan 400.
- **Parameter path** (:id, :teamId, dan sejenisnya) diperiksa dengan ParseIntPipe agar hanya angka yang diterima.

Enum

Enum	Nilai
MemberRole	admin, manager, member
TaskStatus	todo, in_progress, review, done
TaskPriority	low, medium, high, urgent

Kode status HTTP

Kode	Arti	Digunakan saat
200	OK	GET, PATCH, DELETE berhasil
201	Created	POST berhasil
400	Bad Request	Validasi DTO gagal atau aturan bisnis dilanggar
404	Not Found	Resource atau parent tidak ditemukan
409	Conflict	Nilai unik duplikat (nama team, email member)
500	Internal Server Error	Kesalahan tak terduga

Format response error (seragam)

```
{
 "statusCode": 400,
 "message": [
  "name must be longer than or equal to 3 characters",
  "name should not be empty"
 ],
 "error": "Bad Request"
}
```

```
{
 "statusCode": 404,
 "message": "Team with id 99 not found",
 "error": "Not Found"
}
```

3.2 Teams API

3.2.1 Create Team

`POST /teams`

Field	Tipe	Validasi Class-Validator
name	string	@IsString() @IsNotEmpty() @MinLength(3) @MaxLength(100)
description	string	@IsOptional() @IsString() @MaxLength(1000)

Contoh request:

```
{
 "name": "Backend Team",
 "description": "Tim pengembang layanan backend"
}
```

Response 201 Created:

```
{
 "id": 1,
 "name": "Backend Team",
 "description": "Tim pengembang layanan backend",
 "created_at": "2026-11-12T08:30:00.000Z",
 "updated_at": "2026-11-12T08:30:00.000Z"
}
```

Error: `400` (validasi), `409` (nama team sudah dipakai).

3.2.2 Read All Teams

`GET /teams` dengan query opsional `page` (default 1) dan `limit` (default 10, maksimum 100), divalidasi `@IsOptional() @Type(() => Number) @IsInt() @Min(1)`.

Response `200 OK`:

```
{
 "data": [
  {
   "id": 1,
   "name": "Backend Team",
   "description": "Tim pengembang layanan backend",
   "created_at": "2026-11-12T08:30:00.000Z",
   "updated_at": "2026-11-12T08:30:00.000Z"
  }
 ],
 "meta": { "total": 1, "page": 1, "limit": 10, "total_pages": 1 }
}
```


3.2.3 Read One Team

`GET /teams/:id`

Response `200 OK` (menyertakan jumlah relasi):


```
{
 "id": 1,
 "name": "Backend Team",
 "description": "Tim pengembang layanan backend",
 "projects_count": 2,
 "members_count": 4,
 "created_at": "2026-11-12T08:30:00.000Z",
 "updated_at": "2026-11-12T08:30:00.000Z"
}
```


Error: `404` jika team tidak ada.

3.2.4 Update Team

`PATCH /teams/:id`. Body sama dengan Create, semua field opsional (`@IsOptional()` ditambahkan pada setiap validator melalui `PartialType(CreateTeamDto)`).


```
{ "description": "Tim backend dan infrastruktur" }
```


Response 200 OK:


```
{
 "id": 1,
 "name": "Backend Team",
 "description": "Tim backend dan infrastruktur",
 "created_at": "2026-11-12T08:30:00.000Z",
 "updated_at": "2026-11-12T09:15:00.000Z"
}
```


Error: `400`, `404`, `409`.

3.2.5 Delete Team

`DELETE /teams/:id`

Response `200 OK`:

```
{ "message": "Team with id 1 deleted successfully" }
```
Error: 404. Seluruh Project, Member, Task, dan Comment terkait ikut terhapus (CASCADE).

3.3 Projects API

Project selalu diakses melalui Team induknya. Jika :teamId tidak ada, API mengembalikan 404.

3.3.1 Create Project

POST /teams/:teamId/projects

Field	Tipe	Validasi Class-Validator
name	string	@IsString() @IsNotEmpty() @MinLength(3) @MaxLength(150)
description	string	@IsOptional() @IsString() @MaxLength(2000)

```
{
 "name": "Aplikasi E-Commerce",
 "description": "Pengembangan REST API toko online"
}
```
Response 201 Created:

```
{
 "id": 1,
 "name": "Aplikasi E-Commerce",
 "description": "Pengembangan REST API toko online",
 "team_id": 1,
 "created_at": "2026-11-12T08:45:00.000Z",
 "updated_at": "2026-11-12T08:45:00.000Z"
}
```
3.3.2 Read All Projects of a Team

GET /teams/:teamId/projects?page=1&limit=10

Response 200 OK:

```
{
 "data": [
   {
    "id": 1,
    "name": "Aplikasi E-Commerce",
    "description": "Pengembangan REST API toko online",
    "team_id": 1,
    "created_at": "2026-11-12T08:45:00.000Z",
    "updated_at": "2026-11-12T08:45:00.000Z"
   }
 ],
 "meta": { "total": 1, "page": 1, "limit": 10, "total_pages": 1 }
}
```

3.3.3 Read One Project

GET /teams/:teamId/projects/:id

Response 200 OK:

```
{
  "id": 1,
  "name": "Aplikasi E-Commerce",
  "description": "Pengembangan REST API toko online",
  "team_id": 1,
  "tasks_count": 5,
  "created_at": "2026-11-12T08:45:00.000Z",
  "updated_at": "2026-11-12T08:45:00.000Z"
}
```

Error: 404 jika project tidak ada atau bukan milik :teamId.

3.3.4 Update Project

PATCH /teams/:teamId/projects/:id. Body: PartialType(CreateProjectDto).

```
{ "name": "E-Commerce API v2" }
```

Response 200 OK:

```
{
  "id": 1,
  "name": "E-Commerce API v2",
  "description": "Pengembangan REST API toko online",
  "team_id": 1,
  "created_at": "2026-11-12T08:45:00.000Z",
  "updated_at": "2026-11-12T09:20:00.000Z"
}
```

3.3.5 Delete Project

DELETE /teams/:teamId/projects/:id

Response 200 OK:

```
{ "message": "Project with id 1 deleted successfully" }
```

3.4 Members API

Member dikelola dalam konteks Team.

3.4.1 Create Member

POST /teams/:teamId/members

Field	Tipe	Validasi Class-Validator
name	string	@IsString() @IsNotEmpty() @MinLength(2) @MaxLength(100)
email	string	@IsEmail() @MaxLength(150)

Field	Tipe	Validasi Class-Validator
role	enum	@IsOptional() @IsEnum(MemberRole) (default member)

```
{
 "name": "Budi Santoso",
 "email": "budi@example.com",
 "role": "manager"
}
```

Response 201 Created:

```
{
 "id": 1,
 "name": "Budi Santoso",
 "email": "budi@example.com",
 "role": "manager",
 "team_id": 1,
 "created_at": "2026-11-12T09:00:00.000Z",
 "updated_at": "2026-11-12T09:00:00.000Z"
}
```

Error: 400, 404 (team tidak ada), 409 (email sudah terdaftar).

3.4.2 Read All Members of a Team

GET /teams/:teamId/members?role=manager&page=1&limit=10. Filter role opsional dengan @IsOptional() @IsEnum(MemberRole).

Response 200 OK:

```
{
 "data": [
   {
    "id": 1,
    "name": "Budi Santoso",
    "email": "budi@example.com",
    "role": "manager",
    "team_id": 1,
    "created_at": "2026-11-12T09:00:00.000Z",
    "updated_at": "2026-11-12T09:00:00.000Z"
   }
 ],
 "meta": { "total": 1, "page": 1, "limit": 10, "total_pages": 1 }
}
```

3.4.3 Read One Member

GET /teams/:teamId/members/:id

Response 200 OK:

```
{
 "id": 1,
```

```
  "name": "Budi Santoso",
  "email": "budi@example.com",
  "role": "manager",
  "team_id": 1,
  "created_at": "2026-11-12T09:00:00.000Z",
  "updated_at": "2026-11-12T09:00:00.000Z"
}
```

3.4.4 Update Member

`PATCH /teams/:teamId/members/:id`. Body: `PartialType(CreateMemberDto)`.

```
{ "role": "admin" }
```

Response 200 OK:

```
{
  "id": 1,
  "name": "Budi Santoso",
  "email": "budi@example.com",
  "role": "admin",
  "team_id": 1,
  "created_at": "2026-11-12T09:00:00.000Z",
  "updated_at": "2026-11-12T09:30:00.000Z"
}
```

3.4.5 Delete Member

`DELETE /teams/:teamId/members/:id`

Response 200 OK:

```
{ "message": "Member with id 1 deleted successfully" }
```

Task yang ditugaskan ke member ini menjadi tidak ter-assign (assigned_member_id = NULL).

3.5 Tasks API

Task diakses melalui Project induknya.

3.5.1 Create Task

`POST /projects/:projectId/tasks`

Field	Tipe	Validasi Class-Validator
title	string	@IsString() @IsNotEmpty() @MinLength(3) @MaxLength(200)
description	string	@IsOptional() @IsString() @MaxLength(5000)
status	enum	@IsOptional() @IsEnum(TaskStatus) (default todo)
priority	enum	@IsOptional() @IsEnum(TaskPriority) (default medium)

Field	Tipe	Validasi Class-Validator
due_date	string	@IsOptional() @IsDateString() (format YYYY-MM-DD)
assigned_member_id	number	@IsOptional() @IsInt() @Min(1)

```
{
 "title": "Implementasi endpoint checkout",
 "description": "Buat endpoint POST /orders beserta validasi stok",
 "status": "todo",
 "priority": "high",
 "due_date": "2026-11-30",
 "assigned_member_id": 1
}
```

Response 201 Created:

```
{
 "id": 1,
 "title": "Implementasi endpoint checkout",
 "description": "Buat endpoint POST /orders beserta validasi stok",
 "status": "todo",
 "priority": "high",
 "due_date": "2026-11-30",
 "project_id": 1,
 "assigned_member_id": 1,
 "created_at": "2026-11-12T10:00:00.000Z",
 "updated_at": "2026-11-12T10:00:00.000Z"
}
```

Error: 400 (validasi, atau member bukan anggota Team pemilik Project), 404 (project atau member tidak ada).

3.5.2 Read All Tasks (Filter Status, Priority, Assignment)

GET /projects/:projectId/tasks

Query	Tipe	Validasi	Fungsi
status	enum	@IsOptional() @IsEnum(TaskStatus)	Filter berdasarkan status
priority	enum	@IsOptional() @IsEnum(TaskPriority)	Filter berdasarkan prioritas
assigned_member_id	number	@IsOptional() @Type(() => Number) @IsInt()	Filter berdasarkan penerima tugas
unassigned	boolean	@IsOptional() @IsBooleanString()	Jika true, tampilkan task tanpa penerima
page, limit	number	@IsOptional() @Type(() =>	Pagination (default 1

Query	Tipe	Validasi	Fungsi
		Number) @IsInt() @Min(1)	dan 10)

Contoh: GET /projects/1/tasks?status=in_progress&assigned_member_id=1&page=1&limit=10

Response 200 OK:

```
{
 "data": [
  {
   "id": 1,
   "title": "Implementasi endpoint checkout",
   "description": "Buat endpoint POST /orders beserta validasi stok",
   "status": "in_progress",
   "priority": "high",
   "due_date": "2026-11-30",
   "project_id": 1,
   "assigned_member_id": 1,
   "assigned_member": { "id": 1, "name": "Budi Santoso", "email": "budi@example.com" },
   "created_at": "2026-11-12T10:00:00.000Z",
   "updated_at": "2026-11-12T11:00:00.000Z"
  }
 ],
 "meta": { "total": 1, "page": 1, "limit": 10, "total_pages": 1 }
}
```

3.5.3 Read One Task

GET /projects/:projectId/tasks/:id

Response 200 OK:

```
{
 "id": 1,
 "title": "Implementasi endpoint checkout",
 "description": "Buat endpoint POST /orders beserta validasi stok",
 "status": "in_progress",
 "priority": "high",
 "due_date": "2026-11-30",
 "project_id": 1,
 "assigned_member_id": 1,
 "assigned_member": { "id": 1, "name": "Budi Santoso", "email": "budi@example.com" },
 "comments_count": 3,
 "created_at": "2026-11-12T10:00:00.000Z",
 "updated_at": "2026-11-12T11:00:00.000Z"
}
```

3.5.4 Update Task

PATCH /projects/:projectId/tasks/:id. Body: PartialType(CreateTaskDto).

```
{ "status": "review", "priority": "urgent" }
```
Response 200 OK:

```
{
  "id": 1,
  "title": "Implementasi endpoint checkout",
  "description": "Buat endpoint POST /orders beserta validasi stok",
  "status": "review",
  "priority": "urgent",
  "due_date": "2026-11-30",
  "project_id": 1,
  "assigned_member_id": 1,
  "created_at": "2026-11-12T10:00:00.000Z",
  "updated_at": "2026-11-12T12:00:00.000Z"
}
```

3.5.5 Assign Task ke Member

`PATCH /projects/:projectId/tasks/:id/assign`

Field	Tipe	Validasi Class-Validator
assigned_member_id	number atau null	@ValidateIf((o) => o.assigned_member_id !== null) @IsInt() @Min(1); nilai null melepas penugasan

```
{ "assigned_member_id": 2 }
```
Response 200 OK:

```
{
  "id": 1,
  "title": "Implementasi endpoint checkout",
  "status": "review",
  "priority": "urgent",
  "project_id": 1,
  "assigned_member_id": 2,
  "assigned_member": { "id": 2, "name": "Siti Aminah", "email": "siti@example.com" },
  "updated_at": "2026-11-12T12:30:00.000Z"
}
```
Error: 400 (member bukan anggota Team pemilik Project), 404 (task atau member tidak ada).

3.5.6 Delete Task

`DELETE /projects/:projectId/tasks/:id`

Response 200 OK:

```
{ "message": "Task with id 1 deleted successfully" }
```

3.6 Comments API

Komentar dikelola melalui Task induknya (menambah dan melihat).

3.6.1 Add Comment

POST /tasks/:taskId/comments

Field	Tipe	Validasi Class-Validator
content	string	@IsString() @IsNotEmpty() @MinLength(1) @MaxLength(2000)
member_id	number	@IsInt() @Min(1)

```
{
 "content": "Validasi stok sudah selesai, siap direview.",
 "member_id": 1
}
```

Response 201 Created:

```
{
 "id": 1,
 "content": "Validasi stok sudah selesai, siap direview.",
 "task_id": 1,
 "member_id": 1,
 "created_at": "2026-11-12T12:45:00.000Z",
 "updated_at": "2026-11-12T12:45:00.000Z"
}
```

Error: 400 (validasi, atau member bukan anggota Team pemilik Project), 404 (task atau member tidak ada).

3.6.2 View Comments of a Task

GET /tasks/:taskId/comments?page=1&limit=20. Urutan dari komentar terlama ke terbaru (created_at ASC).

Response 200 OK:

```
{
 "data": [
   {
    "id": 1,
    "content": "Validasi stok sudah selesai, siap direview.",
    "task_id": 1,
    "member_id": 1,
    "member": { "id": 1, "name": "Budi Santoso", "role": "manager" },
    "created_at": "2026-11-12T12:45:00.000Z",
    "updated_at": "2026-11-12T12:45:00.000Z"
   }
 ],
 "meta": { "total": 1, "page": 1, "limit": 20, "total_pages": 1 }
}
```

3.7 Ringkasan Seluruh Endpoint

Modul	Method	Endpoint	Fungsi	Status Sukses
Teams	POST	/teams	Membuat team	201
Teams	GET	/teams	Daftar team	200
Teams	GET	/teams/:id	Detail team	200
Teams	PATCH	/teams/:id	Memperbarui team	200
Teams	DELETE	/teams/:id	Menghapus team	200
Projects	POST	/teams/:teamId/projects	Membuat project	201
Projects	GET	/teams/:teamId/projects	Daftar project team	200
Projects	GET	/teams/:teamId/projects/:id	Detail project	200
Projects	PATCH	/teams/:teamId/projects/:id	Memperbarui project	200
Projects	DELETE	/teams/:teamId/projects/:id	Menghapus project	200
Members	POST	/teams/:teamId/members	Menambah member	201
Members	GET	/teams/:teamId/members	Daftar member team	200
Members	GET	/teams/:teamId/members/:id	Detail member	200
Members	PATCH	/teams/:teamId/members/:id	Memperbarui member	200
Members	DELETE	/teams/:teamId/members/:id	Menghapus member	200
Tasks	POST	/projects/:projectId/tasks	Membuat task	201
Tasks	GET	/projects/:projectId/tasks	Daftar task (filter)	200
Tasks	GET	/projects/:projectId/tasks/:id	Detail task	200
Tasks	PATCH	/projects/:projectId/tasks/:id	Memperbarui task	200
Tasks	PATCH	/projects/:projectId/tasks/:id /assign	Assign task ke member	200
Tasks	DELETE	/projects/:projectId/tasks/:id	Menghapus task	200
Comments	POST	/tasks/:taskId/comments	Menambah komentar	201
Comments	GET	/tasks/:taskId/comments	Melihat komentar	200

Modul	Method	Endpoint	Fungsi	Status Sukses
			task	

4. Panduan Instalasi & Menjalankan Proyek di Localhost

4.1 Prasyarat

Perangkat Lunak	Versi Minimum	Perintah Pengecekan
Node.js	18 LTS	node -v
npm	9	npm -v
MySQL Server	8.0	mysql --version
Git	2.x	git --version
NestJS CLI (opsional)	10	nest --version

4.2 Langkah 1: Clone Repository dan Instal Dependensi

```
# 1. Clone repository
git clone https://github.com/<username>/task-management-system.git
cd task-management-system

# 2. Instal seluruh dependensi proyek
npm install
```

Ganti <username> dengan nama akun GitHub Anda. Jika membuat proyek dari nol, jalankan perintah berikut:

```
npm i -g @nestjs/cli
nest new task-management-system
cd task-management-system

npm install @nestjs/typeorm typeorm mysql2 @nestjs/config class-validator class-transformer

nest g resource teams --no-spec
nest g resource projects --no-spec
nest g resource members --no-spec
nest g resource tasks --no-spec
nest g resource comments --no-spec
```

4.3 Langkah 2: Membuat Database MySQL

```
mysql -u root -p
```

```
CREATE DATABASE task_management
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

```
SHOW DATABASES;
EXIT;
```

4.4 Langkah 3: Konfigurasi File .env

Buat file .env di root proyek (dan jangan di-commit; tambahkan ke .gitignore):

```
cp .env.example .env
```

Isi .env:

```
PORT=3000

DB_HOST=localhost
DB_PORT=3306
DB_USERNAME=root
DB_PASSWORD=your_mysql_password
DB_NAME=task_management
DB_SYNCHRONIZE=true
```

Catatan: DB_SYNCHRONIZE=true hanya untuk pengembangan lokal. Pada produksi gunakan false dan kelola skema melalui migrasi.

4.5 Langkah 4: Konfigurasi Aplikasi

src/app.module.ts:

```
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { TeamsModule } from './teams/teams.module';
import { ProjectsModule } from './projects/projects.module';
import { MembersModule } from './members/members.module';
import { TasksModule } from './tasks/tasks.module';
import { CommentsModule } from './comments/comments.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'mysql',
        host: config.get<string>('DB_HOST'),
        port: config.get<number>('DB_PORT'),
        username: config.get<string>('DB_USERNAME'),
        password: config.get<string>('DB_PASSWORD'),
        database: config.get<string>('DB_NAME'),
        autoLoadEntities: true,
        synchronize: config.get<string>('DB_SYNCHRONIZE') === 'true',
```

```
    }),
  }),
  TeamsModule,
  ProjectsModule,
  MembersModule,
  TasksModule,
  CommentsModule,
 ],
})
export class AppModule {}
```

src/main.ts:

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';

async function bootstrap() {
 const app = await NestFactory.create(AppModule);

 app.setGlobalPrefix('api/v1');
 app.useGlobalPipes(
  new ValidationPipe({
   whitelist: true,
   forbidNonWhitelisted: true,
   transform: true,
  }),
 );

 await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Contoh entity src/tasks/entities/task.entity.ts:

```
import {
 Column, CreateDateColumn, Entity, JoinColumn, ManyToOne,
 OneToMany, PrimaryGeneratedColumn, UpdateDateColumn,
} from 'typeorm';
import { Project } from '../../projects/entities/project.entity';
import { Member } from '../../members/entities/member.entity';
import { Comment } from '../../comments/entities/comment.entity';

export enum TaskStatus { TODO = 'todo', IN_PROGRESS = 'in_progress', REVIEW = 'review',
DONE = 'done' }
export enum TaskPriority { LOW = 'low', MEDIUM = 'medium', HIGH = 'high', URGENT =
'urgent' }

@Entity('tasks')
```

```
export class Task {
  @PrimaryGeneratedColumn({ unsigned: true })
  id: number;

  @Column({ length: 200 })
  title: string;

  @Column({ type: 'text', nullable: true })
  description: string | null;

  @Column({ type: 'enum', enum: TaskStatus, default: TaskStatus.TODO })
  status: TaskStatus;

  @Column({ type: 'enum', enum: TaskPriority, default: TaskPriority.MEDIUM })
  priority: TaskPriority;

  @Column({ type: 'date', nullable: true })
  due_date: string | null;

  @Column({ unsigned: true })
  project_id: number;

  @ManyToOne(() => Project, (project) => project.tasks, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'project_id' })
  project: Project;

  @Column({ unsigned: true, nullable: true })
  assigned_member_id: number | null;

  @ManyToOne(() => Member, (member) => member.tasks, { onDelete: 'SET NULL', nullable:
true })
  @JoinColumn({ name: 'assigned_member_id' })
  assigned_member: Member | null;

  @OneToMany(() => Comment, (comment) => comment.task)
  comments: Comment[];

  @CreateDateColumn() created_at: Date;
  @UpdateDateColumn() updated_at: Date;
}
```

Contoh DTO src/tasks/dto/create-task.dto.ts:

```
import { IsDateString, IsEnum, IsInt, IsNotEmpty, IsOptional, IsString, MaxLength, Min,
MinLength } from 'class-validator';
import { TaskPriority, TaskStatus } from '../entities/task.entity';

export class CreateTaskDto {
```

```
  @IsString() @IsNotEmpty() @MinLength(3) @MaxLength(200)
  title: string;

  @IsOptional() @IsString() @MaxLength(5000)
  description?: string;

  @IsOptional() @IsEnum(TaskStatus)
  status?: TaskStatus;

  @IsOptional() @IsEnum(TaskPriority)
  priority?: TaskPriority;

  @IsOptional() @IsDateString()
  due_date?: string;

  @IsOptional() @IsInt() @Min(1)
  assigned_member_id?: number;
}
```

4.6 Langkah 5: Sinkronisasi Skema atau Migrasi TypeORM

Opsi A: Sinkronisasi otomatis (disarankan untuk tugas kuliah dan pengembangan lokal). Dengan DB_SYNCHRONIZE=true, TypeORM membuat kelima tabel secara otomatis saat aplikasi pertama kali dijalankan. Tidak ada perintah tambahan.

Opsi B: Migrasi (disarankan untuk produksi). Buat src/database/data-source.ts:

```
import 'dotenv/config';
import { DataSource } from 'typeorm';

export default new DataSource({
  type: 'mysql',
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  username: process.env.DB_USERNAME,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  entities: ['src/**/*.entity.ts'],
  migrations: ['src/database/migrations/*.ts'],
});
```

Tambahkan script pada package.json:

```
{
  "scripts": {
    "typeorm": "typeorm-ts-node-commonjs -d src/database/data-source.ts",
    "migration:generate": "npm run typeorm -- migration:generate src/database/migrations/Init",
    "migration:run": "npm run typeorm -- migration:run",
    "migration:revert": "npm run typeorm -- migration:revert"
```

```
  }
}
```

Jalankan migrasi (set DB_SYNCHRONIZE=false terlebih dahulu):

```
npm run migration:generate
npm run migration:run
```

4.7 Langkah 6: Menjalankan Aplikasi

```
# Mode pengembangan (hot reload)
npm run start:dev

# Mode produksi
npm run build
npm run start:prod
```

Jika berhasil, terminal menampilkan log Nest application successfully started dan API aktif di http://localhost:3000/api/v1.

4.8 Langkah 7: Pengujian Cepat dengan cURL

```
# Buat team
curl -X POST http://localhost:3000/api/v1/teams \
  -H "Content-Type: application/json" \
  -d '{"name":"Backend Team","description":"Tim pengembang layanan backend"}'

# Tambah member
curl -X POST http://localhost:3000/api/v1/teams/1/members \
  -H "Content-Type: application/json" \
  -d '{"name":"Budi Santoso","email":"budi@example.com","role":"manager"}'

# Buat project
curl -X POST http://localhost:3000/api/v1/teams/1/projects \
  -H "Content-Type: application/json" \
  -d '{"name":"Aplikasi E-Commerce"}'

# Buat task
curl -X POST http://localhost:3000/api/v1/projects/1/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Implementasi endpoint checkout","priority":"high","assigned_member_id":1}'

# Filter task
curl "http://localhost:3000/api/v1/projects/1/tasks?status=todo&assigned_member_id=1"

# Tambah komentar
curl -X POST http://localhost:3000/api/v1/tasks/1/comments \
  -H "Content-Type: application/json" \
  -d '{"content":"Mulai dikerjakan hari ini.","member_id":1}'
```

Anda juga dapat mengimpor endpoint ke Postman atau Insomnia dengan base URL yang sama.

4.9 Pemecahan Masalah Umum

Gejala	Penyebab	Solusi
ER_ACCESS_DENIED_ERROR	Username atau password MySQL salah	Periksa DB_USERNAME dan DB_PASSWORD di .env
ER_BAD_DB_ERROR: Unknown database	Database belum dibuat	Jalankan CREATE DATABASE task_management;
ECONNREFUSED 127.0.0.1:3306	MySQL belum berjalan	Jalankan service MySQL (sudo systemctl start mysql, atau XAMPP/Laragon)
EADDRINUSE: port 3000	Port dipakai aplikasi lain	Ubah PORT di .env atau hentikan proses lain
400 property ... should not exist	Field di luar DTO dikirim	Kirim hanya field yang terdaftar pada spesifikasi
Tabel tidak terbentuk	DB_SYNCHRONIZE bernilai false	Set true untuk lokal, atau jalankan npm run migration:run

Penutup

Dokumen ini telah menguraikan arsitektur sistem, rancangan lima tabel database beserta ERD, 23 endpoint API lengkap dengan aturan validasi dan contoh response, serta langkah menjalankan proyek di localhost. Dengan panduan ini, setiap anggota tim dapat memahami struktur sistem dan menjalankannya dari awal.